Solution: Find the Index of the First Occurrence in a String

Problem Statement

Given two strings `str1` and `str2`, return the index of first occurrences of the `str2` string in `str1`. If `str2` doesn't exists in `str1` string, return `-1`.

Examples

- **Example 1**:
  - Input: `str1 = "algorithms", str2 = "rith"`
  - Expected Output: `4`
  - Justification: The substring `"rith"` begins at index **4** in the string `"algorithms"`.
- **Example 2**:
  - Input: `str1 = "openai", str2 = "ai"`
  - Expected Output: `4`
  - Justification: The substring `"ai"` starts at index **4** in the string `"openai"`.
- **Example 3**:
  - Input: `str1 = "challenge", str2 = "test"`
  - Expected Output: `-1`
  - Justification: The substring `"test"` does not appear in the string `"challenge"`, hence the output is **-1**.

Solution

To solve this problem, we'll adopt a straightforward search approach. We iterate through the main string (str1) and for each position, we check if the following characters match the target substring (str2).

This method is effective because it directly compares segments of the str1 with the str2, ensuring that no possible starting point is overlooked. The key to this approach's effectiveness lies in its meticulous examination of each potential match point, guaranteeing that as soon as the str2 is found, its starting index is immediately returned, optimizing for both clarity and efficiency.

Step-by-step Algorithm

- Start by checking if the str2 is empty. If it is, return 0, as an empty string is considered to be found at the beginning.
- Loop through the str1 string starting from index 0.
  - For each index in the str1, compare the substring that starts at this index and spans the length of the str2 with the str2 itself.
  - If a match is found, return the current index as the starting point of the str2 in the str1.
- If no match is found by the end of the loop, return -1 to indicate the str2 was not found in the str1.

Algorithm Walkthrough

Using the input `str1 = "algorithms", str2 = "rith"`:

1. **Initialization:** We start with `str1 = "algorithms"` and `str2 = "rith"`. The goal is to find the first occurrence of `str2` in `str1`.

2. **Check for Empty `str2`:** First, check if `str2` is empty. Since `str2` is not empty, we proceed to the next step.

3. **Start Looping Through `str1`:**
   - We initiate a loop that starts from index 0 of `str1` and will go up to `str1.length - str2.length` to ensure we can compare the full length of `str2` without going out of bounds.

4. **Index 0 (`i = 0`):**
   - Compare `str1.substring(0, 4)` ("algo") with `str2` ("rith").
   - The substring does not match `str2`. Move to the next index.

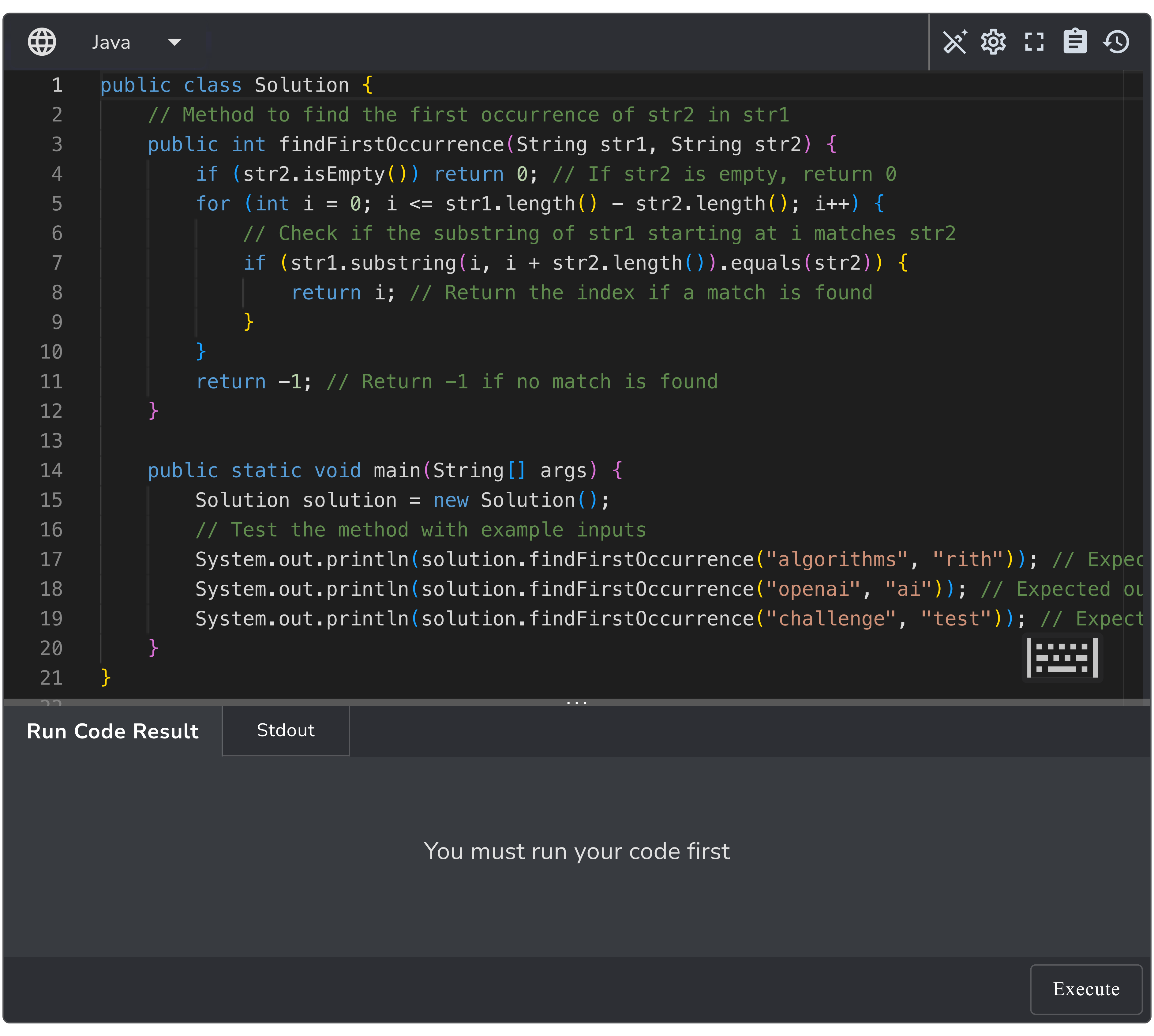
5. **Index 1 (`i = 1`):**
   - Compare `str1.substring(1, 5)` ("lgor") with `str2`.
   - The substring does not match `str2`. Move to the next index.

6. **Continue Looping Until Index 4:**
   - At each index, we are slicing `str1` from the current index `i` to `i + str2.length` and comparing it with `str2`.

7. **Index 4 (`i = 4`):**
   - At index 4, the substring of `str1` is `str1.substring(4, 8)` ("rith").
   - This substring matches `str2` exactly.

8. **Match Found:**
   - Since the substring starting from index 4 matches `str2`, we return **4** as the index where `str2` first occurs in `str1`.

Code



```java
public class Solution {
    // Method to find the first occurrence of str2 in str1
    public int findFirstOccurrence(String str1, String str2) {
        if (str2.isEmpty()) return 0; // If str2 is empty, return 0
        for (int i = 0; i <= str1.length() - str2.length(); i++) {
            // Check if the substring of str1 starting at i matches str2
            if (str1.substring(i, i + str2.length()).equals(str2)) {
                return i; // Return the index if a match is found
            }
        }
        return -1; // Return -1 if no match is found
    }

    public static void main(String[] args) {
        Solution solution = new Solution();
        // Test the method with example inputs
        System.out.println(solution.findFirstOccurrence("algorithms", "rith")); // Expec
        System.out.println(solution.findFirstOccurrence("openai", "ai")); // Expected ou
        System.out.println(solution.findFirstOccurrence("challenge", "test")); // Expect
    }
}
```

You must run your code first

Complexity Analysis

`

- **Time Complexity**: $O((n-m)*m)$, where `n` is the length of the str1 and `m` is the length of the str2. In the worst case, we need to check each substring of the str1 that has the same length as the str2.
- **Space Complexity**: $O(1)$, as we only use a few variables for iteration and comparison, regardless of the input size.

← Previous
Find The Index Of The First Occurrence In A String (Easy)

Next →
Merge Two Sorted Lists (Easy)

☐ Mark as Completed